

LabVIEW Formula Node Technical Documentation

Document Purpose:	Line-by-Line Code Analysis & Architectural Review
Prepared By:	Kiran
Target Platform:	LabVIEW Formula Node (C-Syntax Embedded Structure)
Date:	May 31, 2026

Section 1: Voltage-to-Stroke Conversion Node

1.1 Algorithm Overview

This core node executes a **Dynamic Parameterized Linear Interpolation** algorithm. It maps an empirical, non-linear analog peak-to-peak voltage feedback input (*vpp_val*) into an absolute physical displacement measurement (*stroke*). The execution logic is split into three strict operational regimes: Lower Boundary Clamping, Upper Boundary Clamping, and a Piecewise Linear Lookup Search. This prevents out-of-bounds array violations and guarantees robust mathematical execution under all runtime signals.

1.2 I/O Variable Definition Interface

Variable Name	Data Type	Direction	Functional Engineering Description
vpp_val	Float / Double	Input	The instantaneous peak-to-peak voltage reading sampled from the transducer hardware. Target value to interpolate.
voltages	Array [12]	Input	Calibrated voltage lookup array. Must be strictly sorted in monotonic ascending order.
distances	Array [12]	Input	Physical stroke displacement calibration points corresponding directly to indexes within the voltage array.
stroke	Float / Double	Output	The final resolved physical stroke output (e.g., in mm or micrometers) passed back to the LabVIEW block diagram.

1.3 Line-by-Line Deep-Dive Analysis

```
int i = 0;
int segments = 11; // 12 points make 11 interpolation segments
```

- **Line 1:** Instantiates a local 32-bit signed integer tracking index variable `i` and pre-initializes its block memory location safely to zero.
- **Line 2:** Establishes a bounded execution envelope constraint by setting `segments = 11`. This specifies that the algorithm processes a fixed compilation space of exactly 12 calibration coordinates (from index 0 to 11), yielding 11 bounded linear regions.

```
// 1. Lower Boundary Clamp
if (vpp_val <= voltages[0]) {
    stroke = distances[0];
}
```

- **Line 5:** Computes an under-range validation boundary condition check. It evaluates whether the incoming physical sensor reading `vpp_val` drops below or matches the absolute minimum calibrated voltage envelope entry at `voltages[0]`.
- **Line 6:** If the condition is met, the node executes a safety saturation routine, overriding any dynamic calculations to hard-clamp the output `stroke` directly to the lowest calibrated physical position milestone (`distances[0]`).

```
// 2. Upper Boundary Clamp
else if (vpp_val >= voltages[segments]) {
    stroke = distances[segments];
}
```

- **Line 9:** Evaluates the mirror over-range validation boundary condition. If the lower boundary check passes, it tests if `vpp_val` is equal to or exceeds the upper absolute limit of the calibrated range at array coordinate index 11 (`voltages[11]`).
- **Line 10:** If verified true, it enforces absolute high-side saturation clamping. It maps the output variable `stroke` immediately to the maximum allowable calibration distance (`distances[11]`), preventing unstable extrapolation.

```
// 3. Dynamic Segment Search Loop
else {
    stroke = distances[0]; // Fallback baseline
```

- **Line 13:** Opens the conditional code block execution branch reserved for valid in-bounds operations, where `voltages[0] < vpp_val < voltages[11]`.
- **Line 14:** Applies a proactive safe-state fallback assignment, forcing `stroke` to match `distances[0]` temporarily. This step guarantees that an initialization value is recorded in output registers even if an unexpected processing error occurs inside the loop.

```
for (i = 0; i < segments; i++) {
    if (vpp_val >= voltages[i] && vpp_val <= voltages[i+1]) {
```

- **Line 15:** Launches a structured sequential array search tracking across bounded index cells, starting from index 0 and incrementing through index 10.
- **Line 16:** Implements a critical contextual verification window check. It checks if the current `vpp_val` rests strictly between the lower bound coordinate `voltages[i]` and the upper bound coordinate `voltages[i+1]` for the current segment.

```
        // Standard linear interpolation formula using array lookups
        stroke = distances[i] + (vpp_val - voltages[i]) * (distances[i+1] -
distances[i]) / (voltages[i+1] - voltages[i]);
        break; // Segment found, safely exit the loop
    }
}
}
```

- **Line 18:** Resolves standard continuous localized mathematical linear interpolation using the formula:

$$stroke = y_i + \delta x \cdot (\Delta y / \Delta x)$$

Where $\delta x = (vpp_val - voltages[i])$, $\Delta y = (distances[i+1] - distances[i])$, and $\Delta x = (voltages[i+1] - voltages[i])$. This maps the raw input relative to its position in the current segment.

- **Line 19:** Fires an explicit `break` statement to immediately terminate the search loop. This optimizes execution timing and avoids wasting CPU cycles once the correct segment has been processed.

Section 2: Explicit Positional-Form PID Engine

2.1 Algorithm Overview

This formula node establishes an optimized, **Explicit Positional-Form PID Controller** engineered to drive high-speed electromagnetic actuation systems (such as a voice coil). The node features advanced industrial safety mechanisms, combining a dynamic **Back-Calculation Anti-Windup Protection Routine** to stabilize integration tracks during physical voltage limits, alongside a **Slew-Rate Limiter** to protect voice coil windings from sudden voltage steps.

2.2 I/O Variable Definition Interface

Variable Name	Data Type	Direction	Functional Engineering Description
target_stroke	Float	Input	The desired reference operational position setpoint.
measured_stroke	Float	Input	Physical process feedback value provided by Section 1.
dt	Float	Input	Deterministic cycle loop execution step time period (Δt) in seconds.
prev_error	Float	Input	Stored tracking error value from the previous execution loop.
integral_accumulator_in	Float	Input	The historic accumulated integration track fed from a shift register.
prev_vc_cmd	Float	Input	The actual system output voltage command applied during the last cycle.
kp, ki, kd	Float	Input	The Proportional, Integral, and Derivative control loop gain constants.
max_step_change	Float	Input	Maximum allowable voltage adjustment step per iteration (Slew rate limit).
max_amplitude	Float	Input	Absolute positive safe hardware operational voltage threshold limit.
min_amplitude	Float	Input	Absolute negative safe hardware operational voltage threshold limit.
startup_cmd	Float	Input	Feedforward bias offset value to establish a baseline control voltage.
vc_cmd / out_vc_cmd	Float	Output	The primary control voltage command output sent to the actuator hardware driver.
error_out / out_error	Float	Output	Current cycle error tracking outputs routed to diagnostic registers.
integral_accumulator_out	Float	Output	The newly updated and windup-corrected historical integration track.

2.3 Line-by-Line Deep-Dive Analysis

```
// 1. Calculate Tracking Error
float current_error = target_stroke - measured_stroke;
error_out = current_error;
```

- **Line 12:** Evaluates the instantaneous tracking error by subtracting the system's real physical feedback location (`measured_stroke`) from the target reference setpoint (`target_stroke`).
- **Line 13:** Assigns this raw tracking error directly to the external node output boundary terminal `error_out` for real-time diagnostic plotting.

```
// 2. Compute Proportional Term
float p_term = kp * current_error;
```

- **Line 16:** Calculates the Proportional corrective action vector (**P**). This scales linearly based on the current system error magnitude multiplied by the proportional gain constant `kp`.

```
// 3. Compute Derivative Term
float d_term = 0.0;
if (dt > 0.0) {
    d_term = kd * ((current_error - prev_error) / dt);
}
```

- **Line 19:** Instantiates the local variable `d_term` and explicitly initializes it to zero.
- **Line 20:** Implements a critical safety zero-division guard check ensuring that the execution time delta is positive (`dt > 0.0`). This blocks invalid mathematical calculations if the loop triggers unevenly or stalls.
- **Line 21:** Approximates the true first-order derivative error rate-of-change by calculating the difference between the current loop error and the previous loop error, dividing by the loop step time, and scaling it by the derivative gain `kd`.

```
// 4. Update and Accumulate Integral Term
float current_integral = integral_accumulator_in + (current_error * dt);
float i_term = ki * current_integral;
```

- **Line 25:** Performs continuous forward Euler integration. It computes the area under the current error curve over the elapsed time step (`current_error * dt`) and updates the long-term historical track `integral_accumulator_in`.
- **Line 26:** Scales the updated integration track history against the integral loop gain coefficient `ki` to generate the final corrective Integral term value `i_term`.

```
// 5. Calculate Ideal Target Command  
float ideal_command = startup_cmd + p_term + i_term + d_term;
```

- **Line 29:** Sums the proportional, integral, and derivative corrective vectors together with the feedforward baseline offset value (startup_cmd) to establish an unconstrained ideal target command.

```
// 6. Anti-Windup Protection (Back-calculation method)
if (ideal_command > max_amplitude) {
    float excess = ideal_command - max_amplitude;
    if (ki > 0.0) {
        current_integral -= excess / ki;
    }
    ideal_command = max_amplitude;
}
```

- **Line 32:** Checks if the unconstrained `ideal_command` output exceeds the physical voltage maximum saturation threshold allowed by the hardware drivers.
- **Line 33:** Determines the exact magnitude of the overshoot saturation error (`excess`).
- **Line 34:** Confirms that an active integral calculation condition is present by checking that the gain coefficient is positive (`ki > 0.0`).
- **Line 35: Back-calculation Dynamic Core:** Subtracts the scaled excess error directly out of the historical `current_integral` accumulator. This dynamically offsets the accumulator, preventing integration windup during actuator saturation.
- **Line 37:** Hard-clamps the `ideal_command` value to the maximum physical safe voltage threshold limit (`max_amplitude`).

```
else if (ideal_command < min_amplitude) {
    float excess = ideal_command - min_amplitude;
    if (ki > 0.0) {
        current_integral -= excess / ki;
    }
    ideal_command = min_amplitude;
}
```

- **Lines 38–44:** Executes a mirror-image back-calculation anti-windup safety check on the negative limit. If the unconstrained target falls below `min_amplitude`, it calculates the negative overshoot distance, subtracts it from the integral accumulator track, and clamps the `ideal_command` directly to the lowest safe voltage boundary.

```
// 7. Apply Loop Rate Limiting (Protects voice coil from large voltage steps)
float raw_adjustment = ideal_command - prev_vc_cmd;
```

- **Line 47:** Evaluates the dynamic voltage rate-of-change step size by calculating the difference between the newly processed unconstrained target value (`ideal_command`) and the actual output command applied during the last loop cycle (`prev_vc_cmd`).


```
if (raw_adjustment > max_step_change) {  
    raw_adjustment = max_step_change;  
} else if (raw_adjustment < -max_step_change) {  
    raw_adjustment = -max_step_change;  
}
```

- **Lines 49–50:** Implements a positive slew-rate limiter check. If the requested adjustment step exceeds the maximum allowed step rate (`max_step_change`), it caps the step change down to that maximum value to protect the voice coil from inductive voltage spikes.
- **Lines 51–52:** Implements a negative slew-rate limiter check. If the dynamic downward step change falls below the negative limit (`-max_step_change`), it caps the step change to that negative limit.

```

float next_vc_cmd = prev_vc_cmd + raw_adjustment;

// Final Output Safety Safeguard Clamping
if (next_vc_cmd > max_amplitude) {
    next_vc_cmd = max_amplitude;
} else if (next_vc_cmd < min_amplitude) {
    next_vc_cmd = min_amplitude;
}

```

- **Line 55:** Reconstructs a rate-limited output command parameter `next_vc_cmd` by applying the bounded step adjustment to the historical baseline value `prev_vc_cmd`.
- **Lines 58–62:** Serves as a final safeguard overlay. Re-runs an absolute safety check on the calculated rate-limited command value to guarantee that it has not breached the hardware physical boundaries, applying standard limits if needed.

```

// 8. Map to Shift Registers for memory persistence
vc_cmd = next_vc_cmd;
out_error = current_error;
integral_accumulator_out = current_integral;
out_vc_cmd = next_vc_cmd;

```

- **Line 65:** Assigns the final verified control value to the primary external block diagram output wire terminal `vc_cmd` to drive the physical DAC electronics.
- **Line 66:** Maps the current tracking error to `out_error`, which connects directly to a LabVIEW loop shift register to serve as the historic reference `prev_error` in the next loop execution cycle.
- **Line 67:** Binds the updated and windup-corrected integral sum `current_integral` to `integral_accumulator_out`, routing it to a shift register for the next loop execution cycle.
- **Line 68:** Outputs the final output command value to `out_vc_cmd`, which updates a shift register tracking the last applied voltage command to serve as `prev_vc_cmd` in the next cycle.